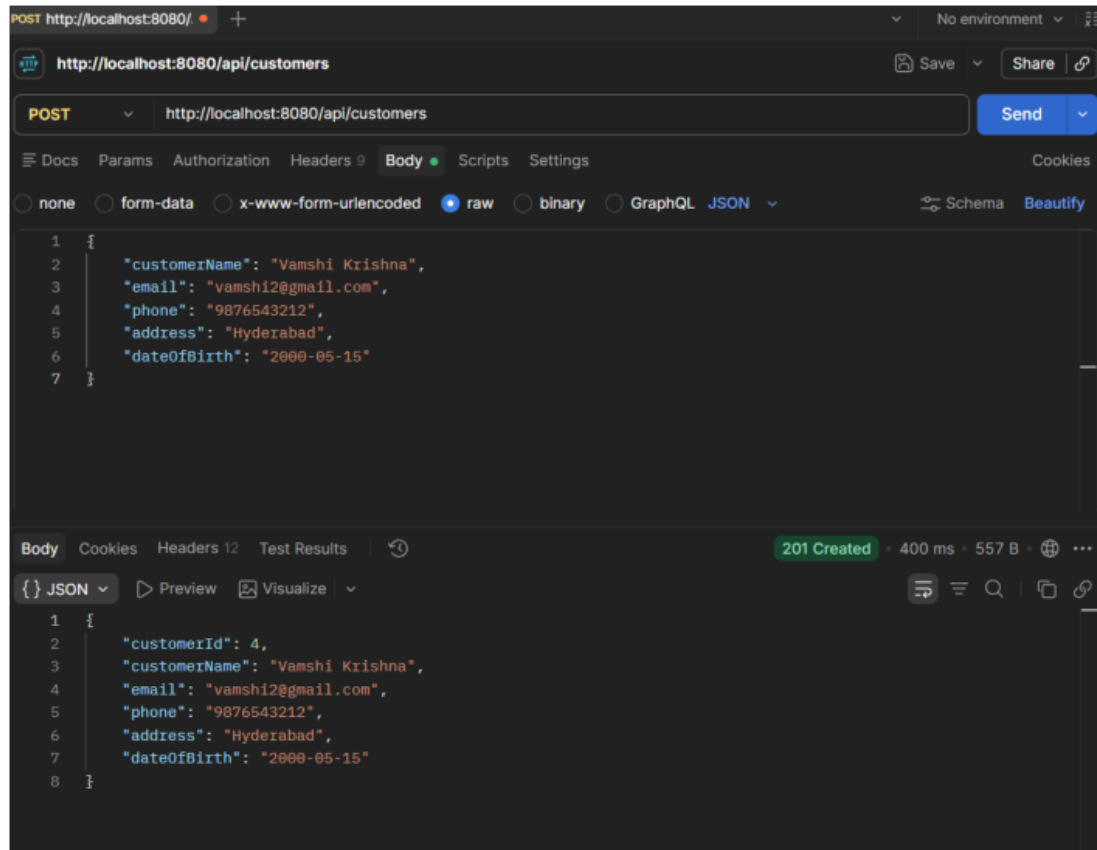


Potman

- **Customer CRUD**

Create Customer

POST: <http://localhost:8080/api/customers>



Database

Customer CRUD – Create Customer

The screenshot shows the POST request used to create a customer through the `/api/customers` API. The request is sent from Postman and the returned customer information demonstrates that the customer record was created successfully. This is the Create operation of Customer CRUD.

```

141
142 • USE finance_db;
143
144 • SELECT * FROM customers;

```

	customer_id	customer_name	email	phone	address	date_of_birth	created_at
▶	1	Vamshi	vamshi@gmail.com	9876543210	Hyderabad	2000-05-15	2026-09-15 19:48:30
	2	Krishna	krishna@gmail.com	9876543211	Bangalore	1999-08-20	2026-09-15 19:48:30
	4	Vamshi Krishna	vamshi2@gmail.com	9876543212	Hyderabad	2000-05-15	2026-09-15 23:41:18
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL

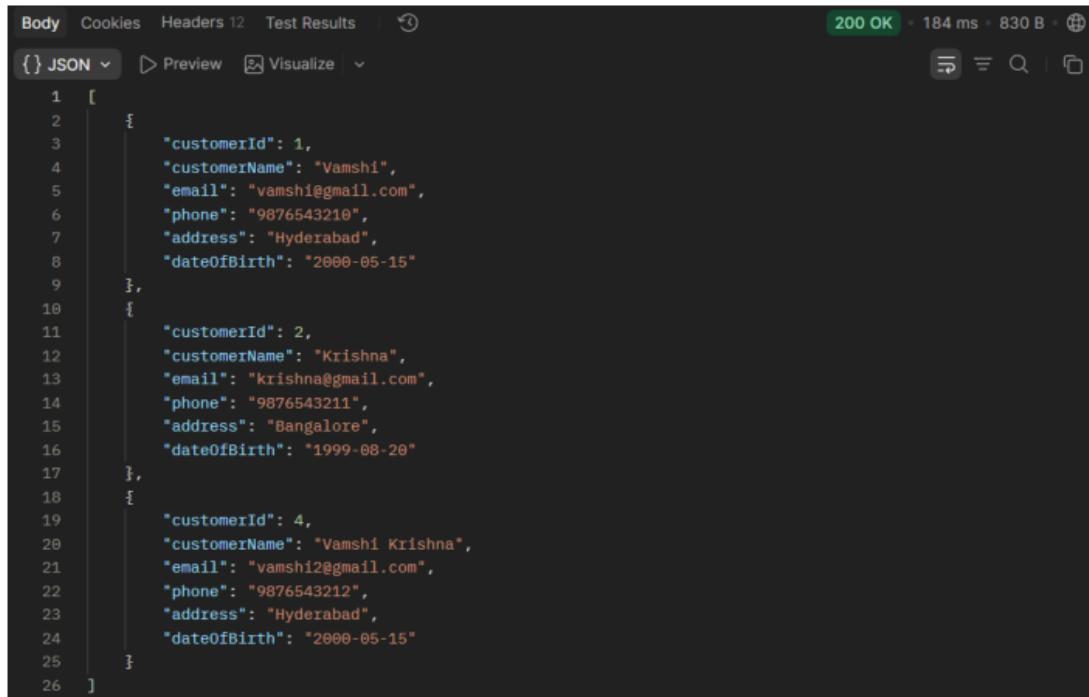
- GET: <http://localhost:8080/api/customers/1>

Body	Cookies	Headers 12	Test Results
200 OK 70 ms 541 B { } JSON Preview Visualize <pre> 1 { 2 "customerId": 1, 3 "customerName": "Vamshi", 4 "email": "vamshi@gmail.com", 5 "phone": "9876543210", 6 "address": "Hyderabad", 7 "dateOfBirth": "2000-05-15" 8 } </pre>			

- GET: <http://localhost:8080/api/customers>

Customer CRUD – Read Customer

The GET requests shown here are used to read customer data. One request retrieves a customer by ID and another retrieves the customer list. These responses demonstrate the Read operation of Customer CRUD.



The screenshot shows a REST client interface with the following details:

- Status:** 200 OK, 184 ms, 830 B
- Body:** JSON
- Response:** A JSON array containing three customer objects.

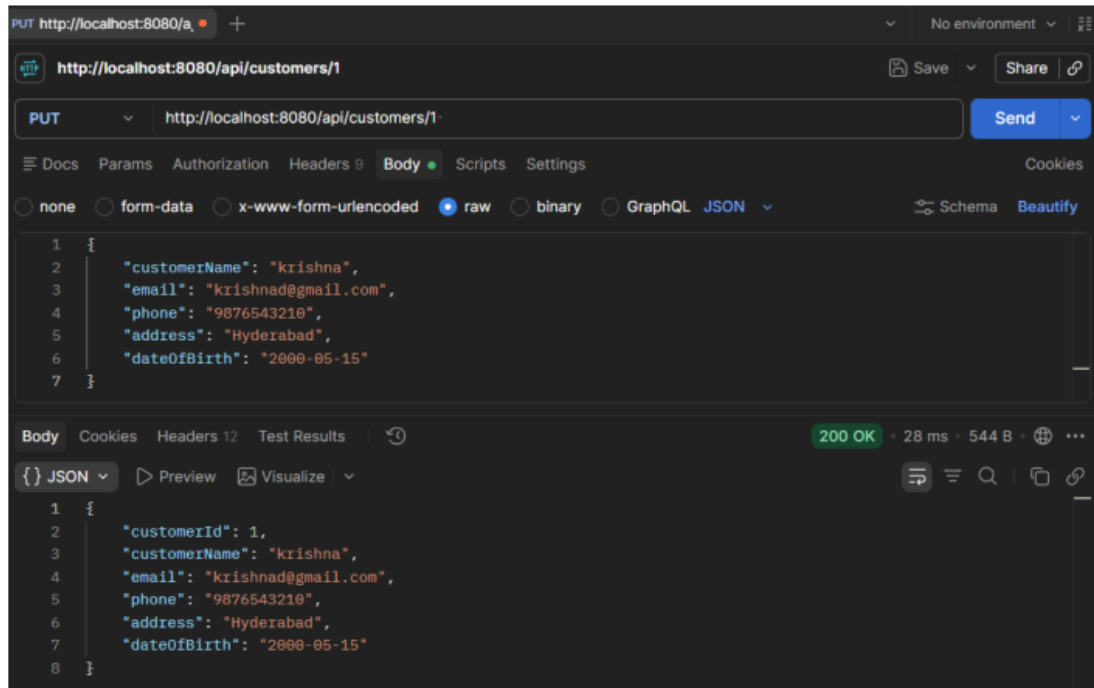
```
1  [
2    {
3      "customerId": 1,
4      "customerName": "Vamshi",
5      "email": "vamshi@gmail.com",
6      "phone": "9876543210",
7      "address": "Hyderabad",
8      "dateOfBirth": "2000-05-15"
9    },
10   {
11     "customerId": 2,
12     "customerName": "Krishna",
13     "email": "krishna@gmail.com",
14     "phone": "9876543211",
15     "address": "Bangalore",
16     "dateOfBirth": "1999-08-20"
17   },
18   {
19     "customerId": 4,
20     "customerName": "Vamshi Krishna",
21     "email": "vamshi2@gmail.com",
22     "phone": "9876543212",
23     "address": "Hyderabad",
24     "dateOfBirth": "2000-05-15"
25   }
26 ]
```

- **Update Customer**

PUT: <http://localhost:8080/api/customers/1>

Customer CRUD – Update Customer

The screenshot shows the PUT request used to update customer information. The response displays the updated customer data returned by the API, demonstrating the Update operation.



- **database**

SELECT * FROM customers;

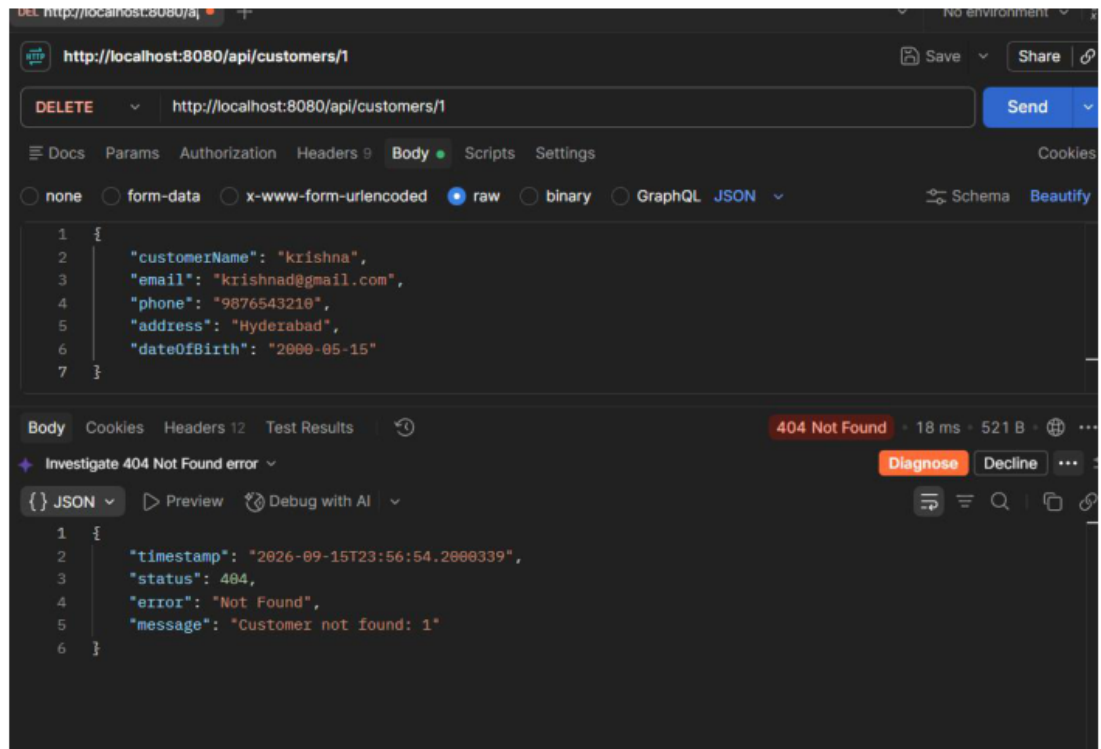
customer_id	customer_name	email	phone	address	date_of_birth	created_at
1	krishna	krishnad@gmail.com	9876543210	Hyderabad	2000-05-15	2026-09-15 19:48:30
2	Krishna	krishna@gmail.com	9876543211	Bangalore	1999-08-20	2026-09-15 19:48:30
4	Vamshi Krishna	vamshi2@gmail.com	9876543212	Hyderabad	2000-05-15	2026-09-15 23:41:18
HULL	HULL	HULL	HULL	HULL	HULL	HULL

- **Delete Customer**

DELETE: <http://localhost:8080/api/customers/1>

Customer CRUD – Delete and Database Verification

The DELETE request removes the selected customer. The MySQL SELECT result shown on the same page is used to verify the database after the deletion. This demonstrates both the Delete operation and database verification.



Database

146 • `SELECT * FROM customers;`

	customer_id	customer_name	email	phone	address	date_of_birth	created_at
▶	2	Krishna	krishna@gmail.com	9876543211	Bangalore	1999-08-20	2026-09-15 19:48:30
	4	Vamshi Krishna	vamshi2@gmail.com	9876543212	Hyderabad	2000-05-15	2026-09-15 23:41:18
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL

• Account CRUD

POST : <http://localhost:8080/api/accounts/1>

```

{
  "accountType": "SAVINGS",
  "balance": 10000
}

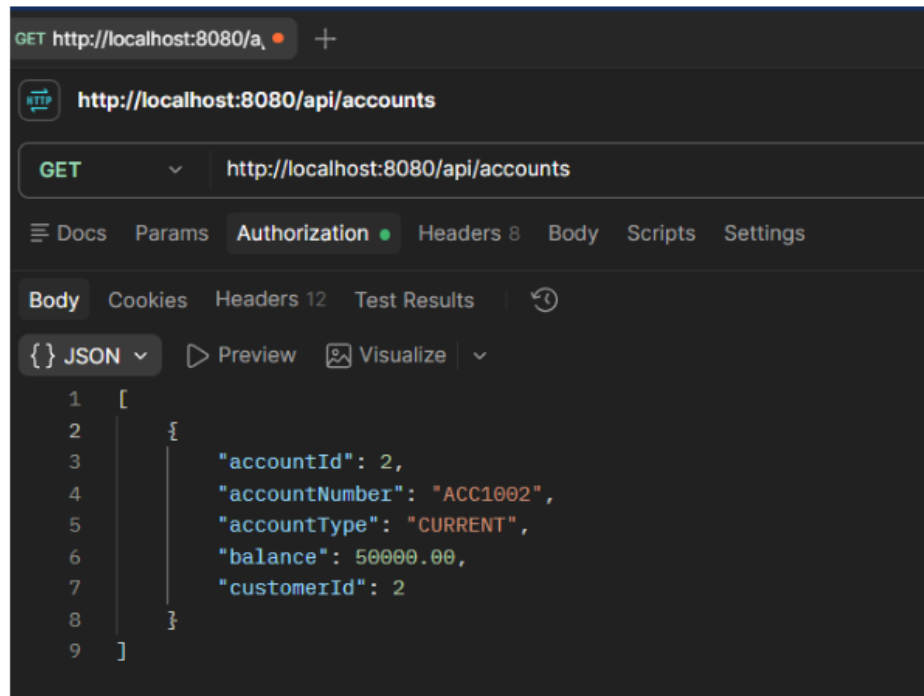
```

get : <http://localhost:8080/api/accounts/2>

Account CRUD – Create Account

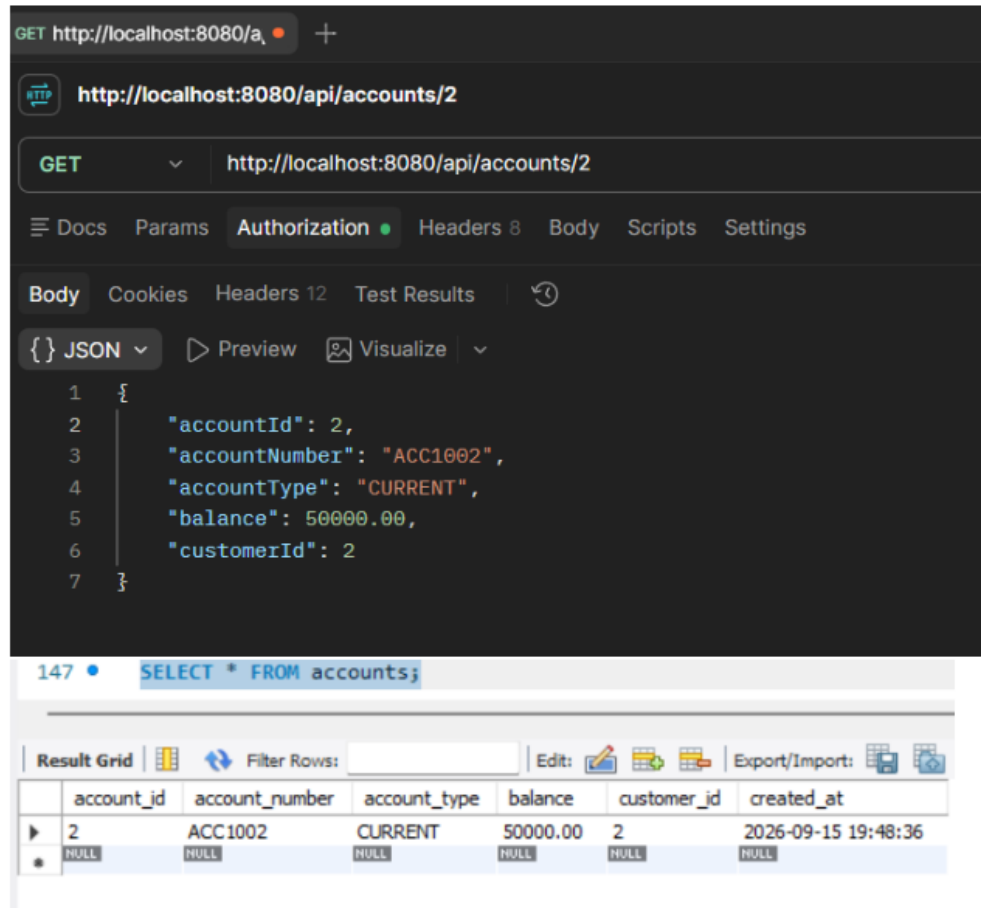
The screenshot documents the POST request used to create an account. The request contains account type and balance information and the account is associated with a customer. The page also begins the account Read testing.

GET <http://localhost:8080/api/accounts>



Account CRUD – Read All Accounts

The GET `/api/accounts` request retrieves account records. The Postman response displays account ID, account number, account type, balance and customer ID, demonstrating the Read operation for accounts.



GET http://localhost:8080/a

http://localhost:8080/api/accounts/2

GET http://localhost:8080/api/accounts/2

Docs Params Authorization Headers 8 Body Scripts Settings

Body Cookies Headers 12 Test Results

JSON Preview Visualize

```

1 {
2   "accountId": 2,
3   "accountNumber": "ACC1002",
4   "accountType": "CURRENT",
5   "balance": 50000.00,
6   "customerId": 2
7 }

```

147 • SELECT * FROM accounts;

Result Grid Filter Rows: Edit: Export/Import:

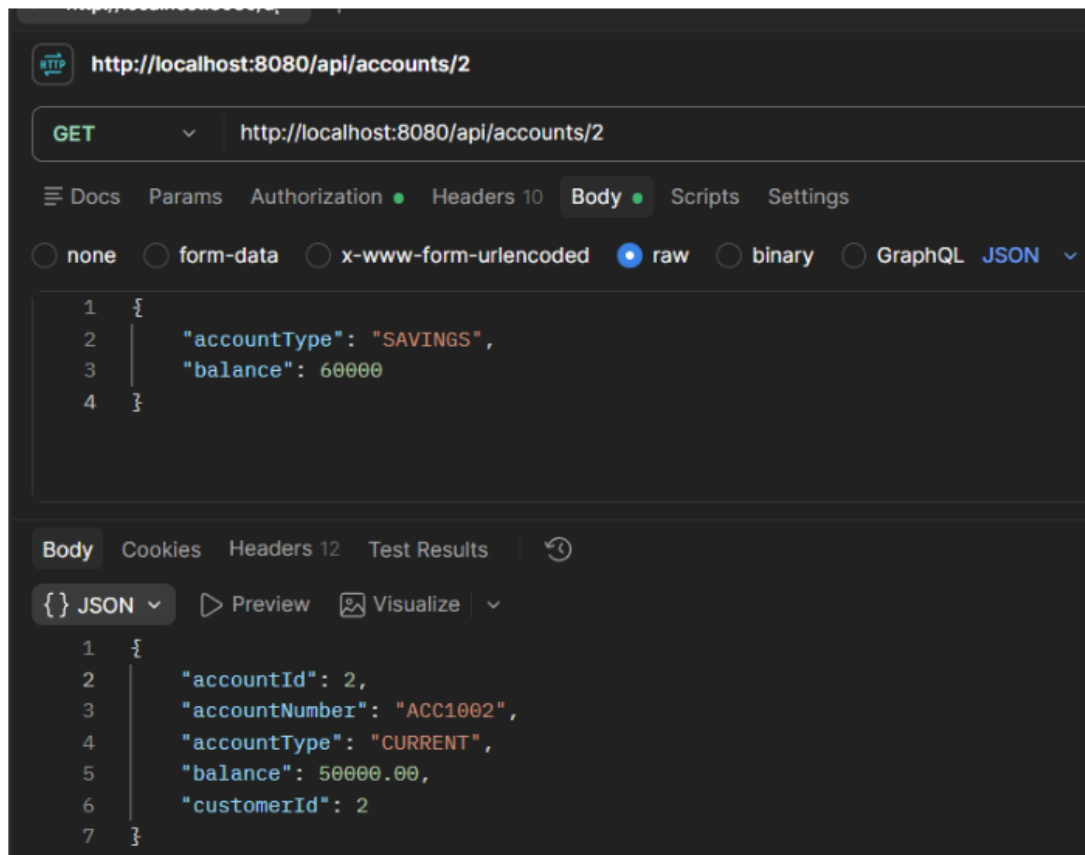
	account_id	account_number	account_type	balance	customer_id	created_at
▶	2	ACC1002	CURRENT	50000.00	2	2026-09-15 19:48:36
•	NULL	NULL	NULL	NULL	NULL	NULL

- **Update Account**

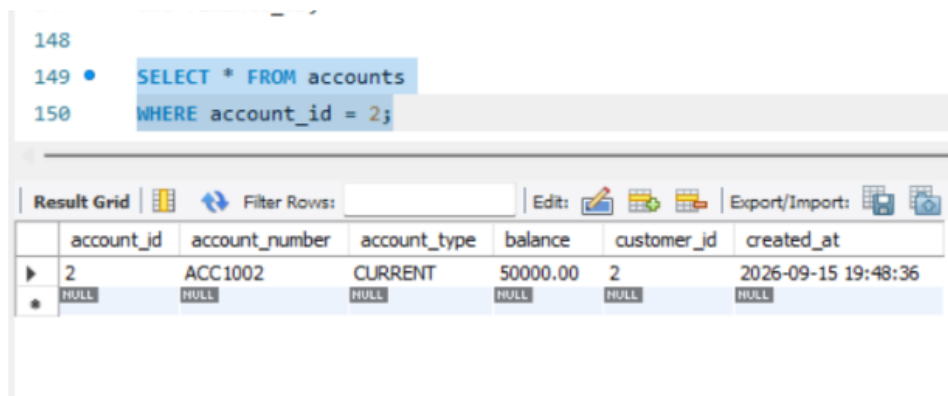
PUT : <http://localhost:8080/api/accounts/2>

Account CRUD – Update Account

The PUT request updates the selected account. The response and database information shown in the project evidence demonstrate that the account details can be modified through the REST API.



- Database



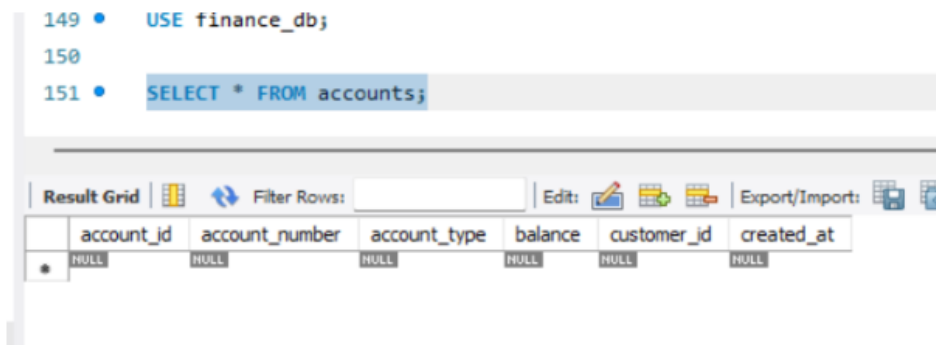
- DELETE Account

DELETE : <http://localhost:8080/api/accounts/2>

database

Account CRUD – Delete and Database Verification

The DELETE request removes the selected account. The database query shown below it is used to check the accounts table after deletion and verify the resulting database state.



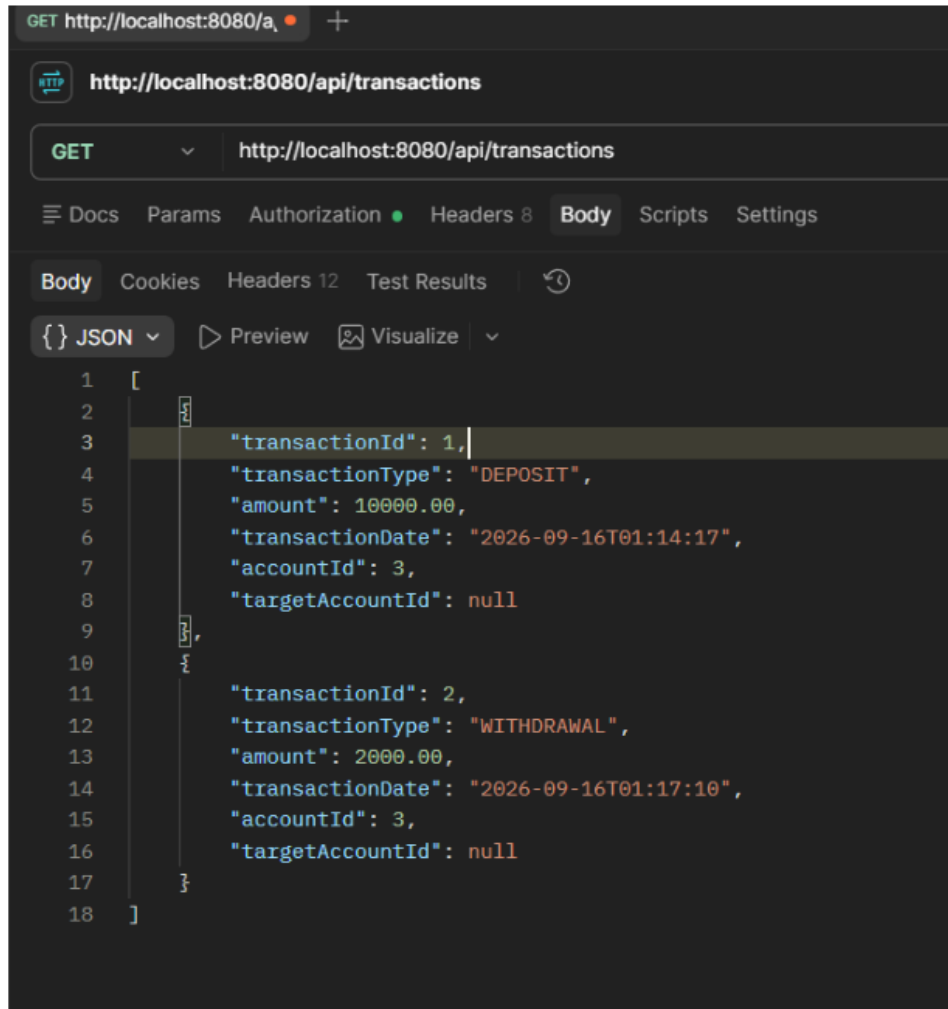
- **Deposit**

POST:

<http://localhost:8080/api/transactions>

Transaction – Deposit

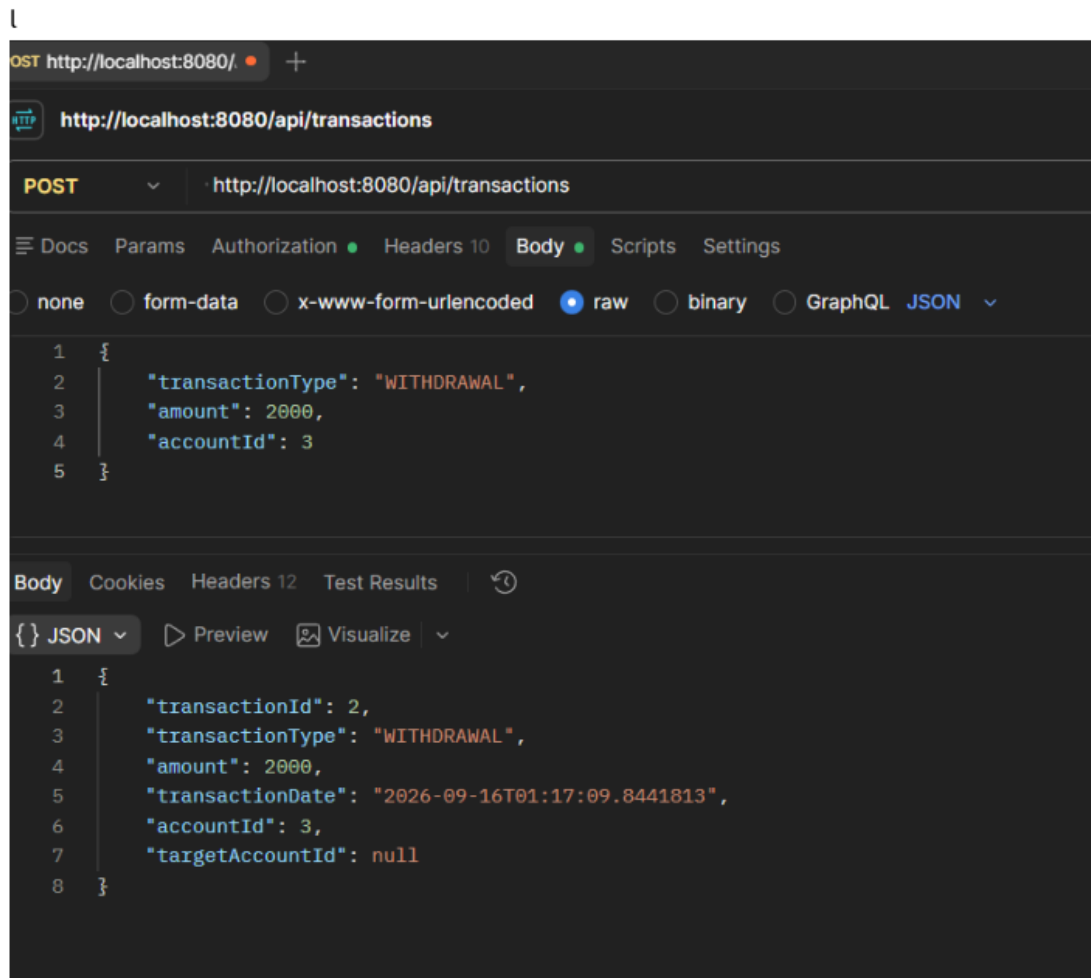
The screenshot introduces the Deposit operation. A POST request is sent to `/api/transactions` to create a deposit transaction for an account. This is part of the financial transaction functionality.



- **Withdrawal**

Transaction – Withdrawal

The screenshot shows the Withdrawal operation using the transaction API. The Postman response records the withdrawal type, amount, transaction date and account ID.

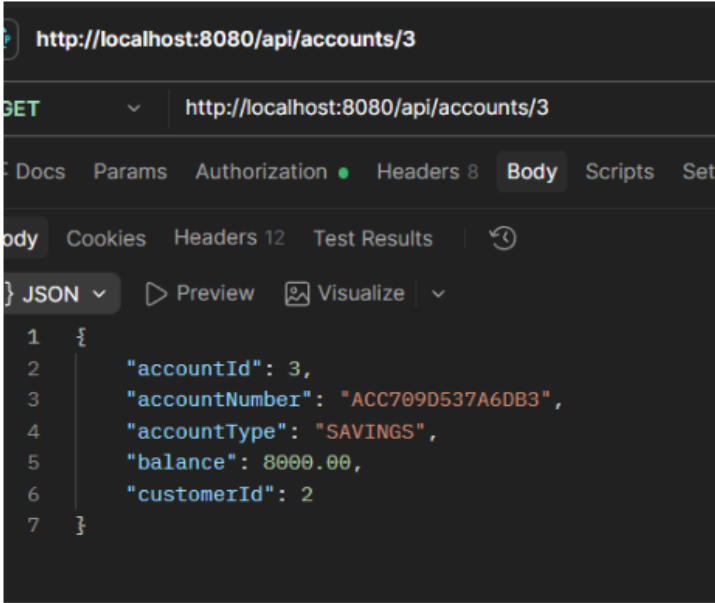


Balance

GET :`http://localhost:8080/api/accounts/3`

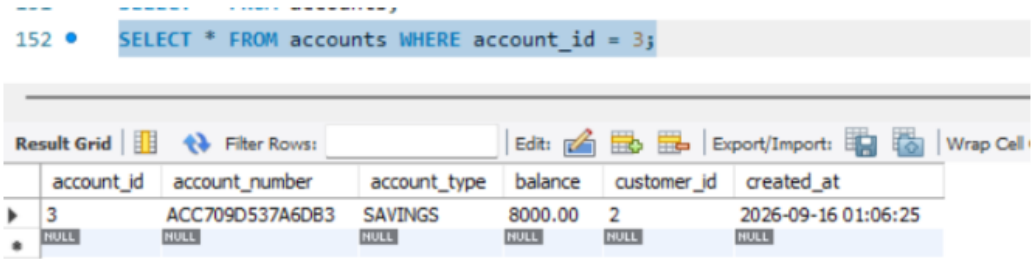
Account Balance Verification

After the deposit and withdrawal operations, the GET `/api/accounts/3` endpoint is used to retrieve the account. The returned balance is used to verify the effect of the transaction operations.



Database

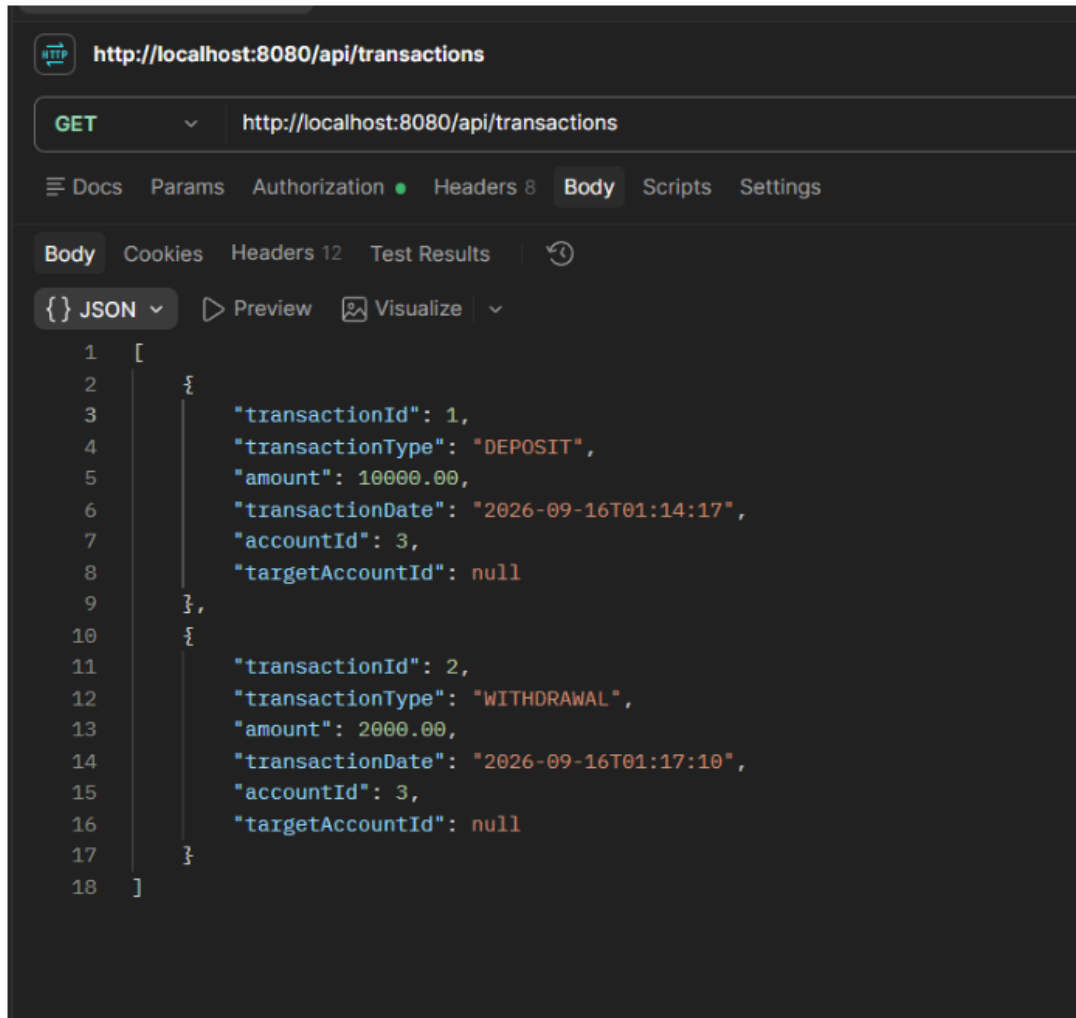
SELECT * FROM accounts WHERE account_id = 3;



- Transaction CRUD

Balance and Database Verification – Transaction CRUD

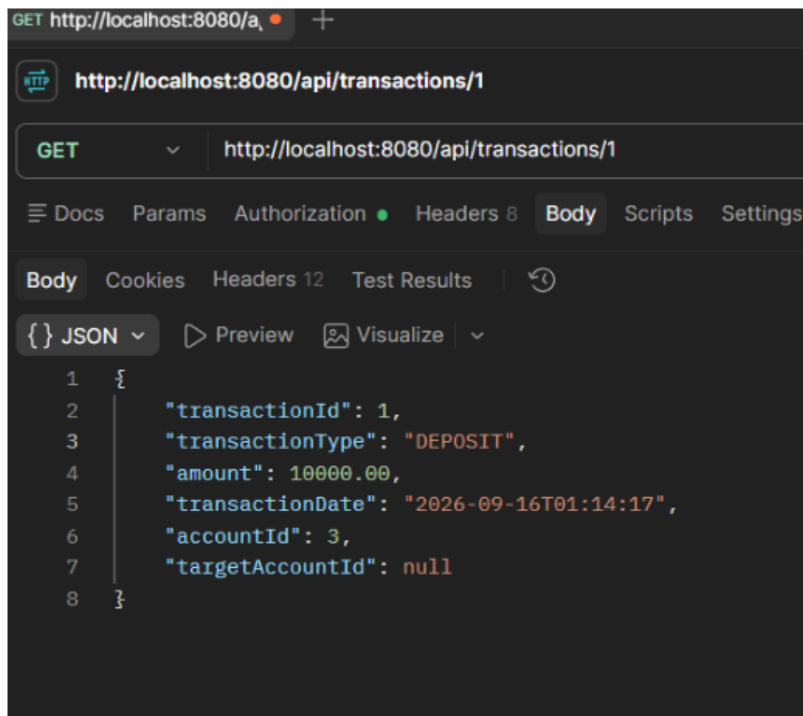
The account response and MySQL SELECT query verify the account balance directly in the database. The page then starts the Transaction CRUD section, connecting the API testing with database verification.



GET: <http://localhost:8080/api/transactions/1>

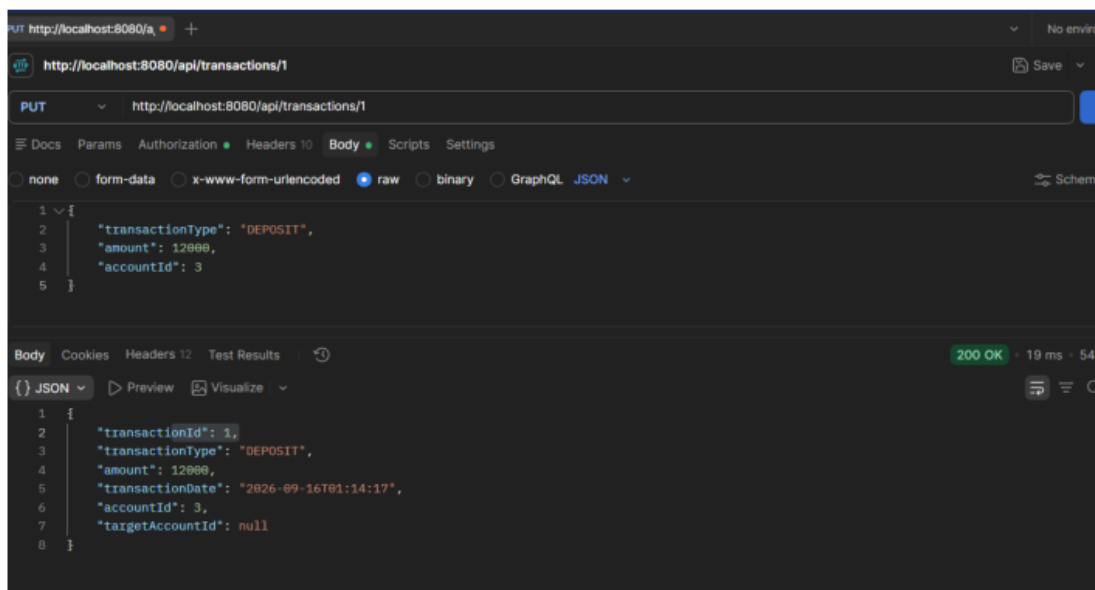
Transaction CRUD – Read

The GET transaction request retrieves transaction information by ID. The API response provides the stored transaction details and demonstrates the Read operation for transactions.



- **Update Transaction**

Get: <http://localhost:8080/api/transactions/1>

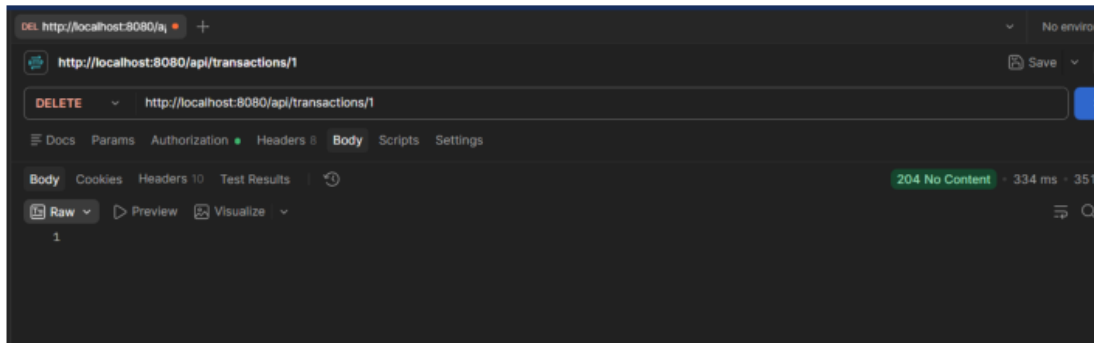


Transaction CRUD – Update

The transaction is retrieved and then updated through the PUT operation. The Postman response demonstrates that the transaction record can be modified through the REST API.

- **DELETE Transaction.**

<http://localhost:8080/api/transactions/1>



- **Database**

```
152 • SELECT * FROM accounts WHERE account_id = 3;
153 • SELECT * FROM transactions;
```

transaction_id	transaction_type	amount	transaction_date	description	account_id	target_account_id
2	WITHDRAWAL	2000.00	2026-09-16 01:17:10		3	

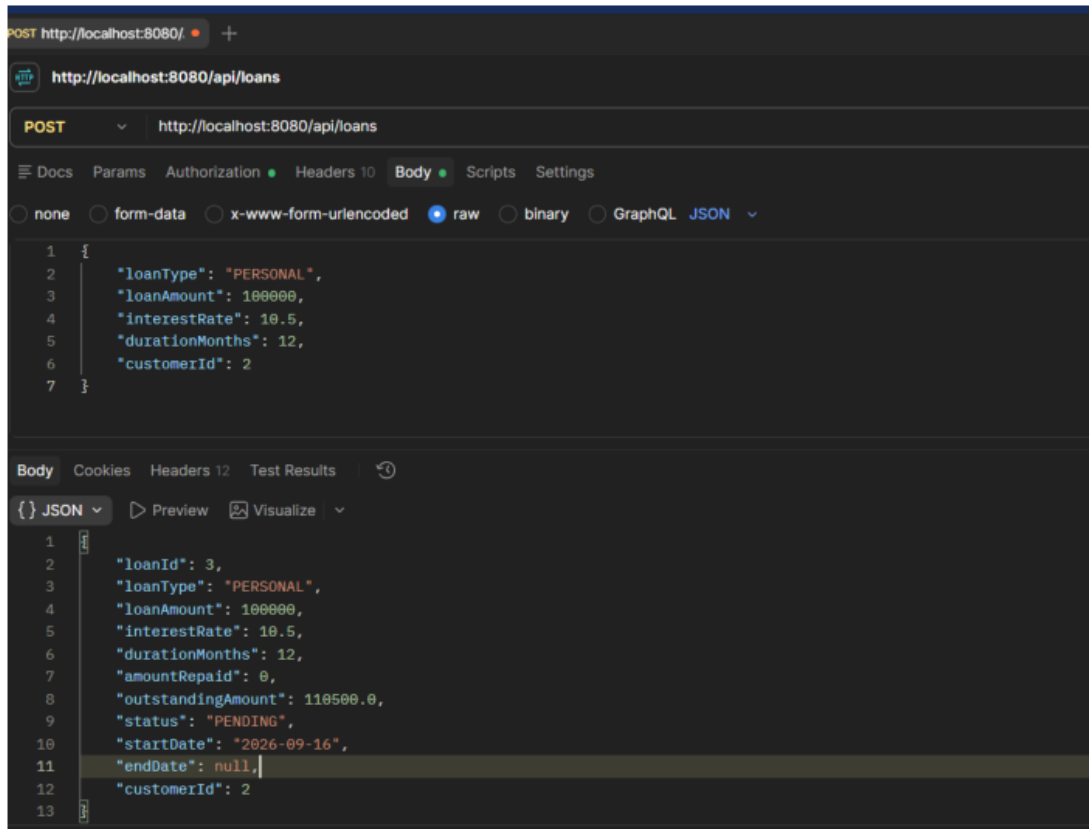
-
- **Loan CRUD**

Create a Loan

POST : <http://localhost:8080/api/loans>

Transaction CRUD – Delete and Loan CRUD

The DELETE request is used to remove a transaction. The page then begins Loan CRUD and shows the endpoint used to create a loan, demonstrating the transition from transaction management to loan management.

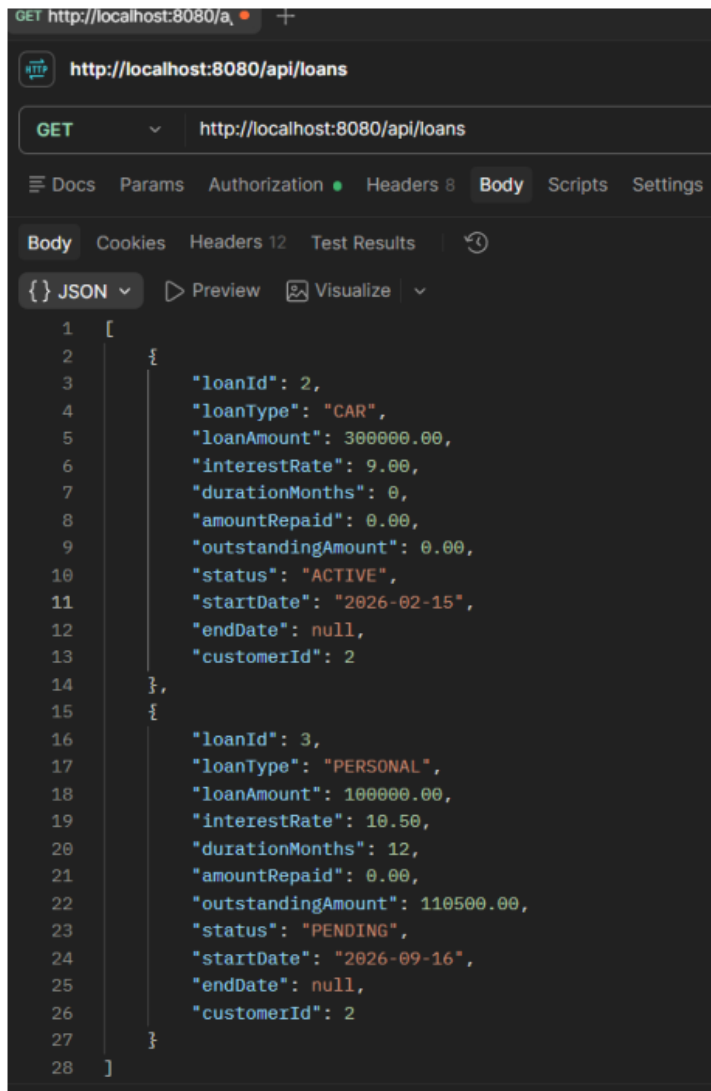


Loan cued (Read)

Get: <http://localhost:8080/api/loans>

Loan CRUD – Create and Read

The POST `/api/loans` request creates a loan using loan type, loan amount, interest rate, duration and customer ID. The GET request retrieves the loan records, demonstrating the Create and Read operations.

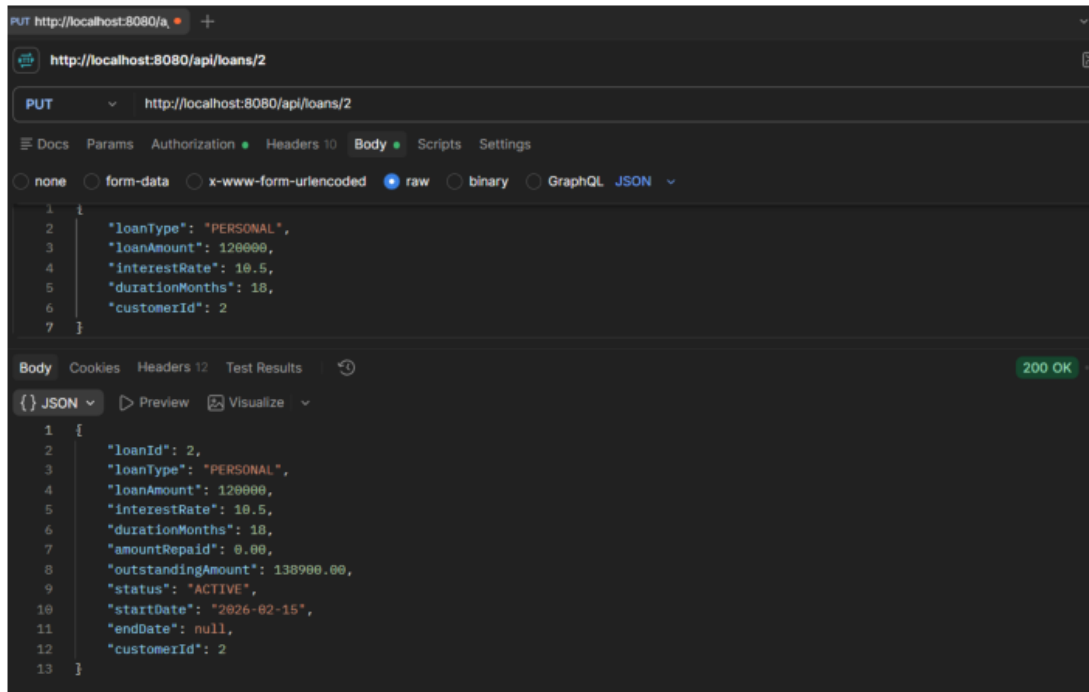


- **Loan Update**

PUT: <http://localhost:8080/api/loans/1>

Loan CRUD – Update

The PUT request updates the selected loan. The Postman response displays the resulting loan details and status, demonstrating the Update operation in the Loan module.



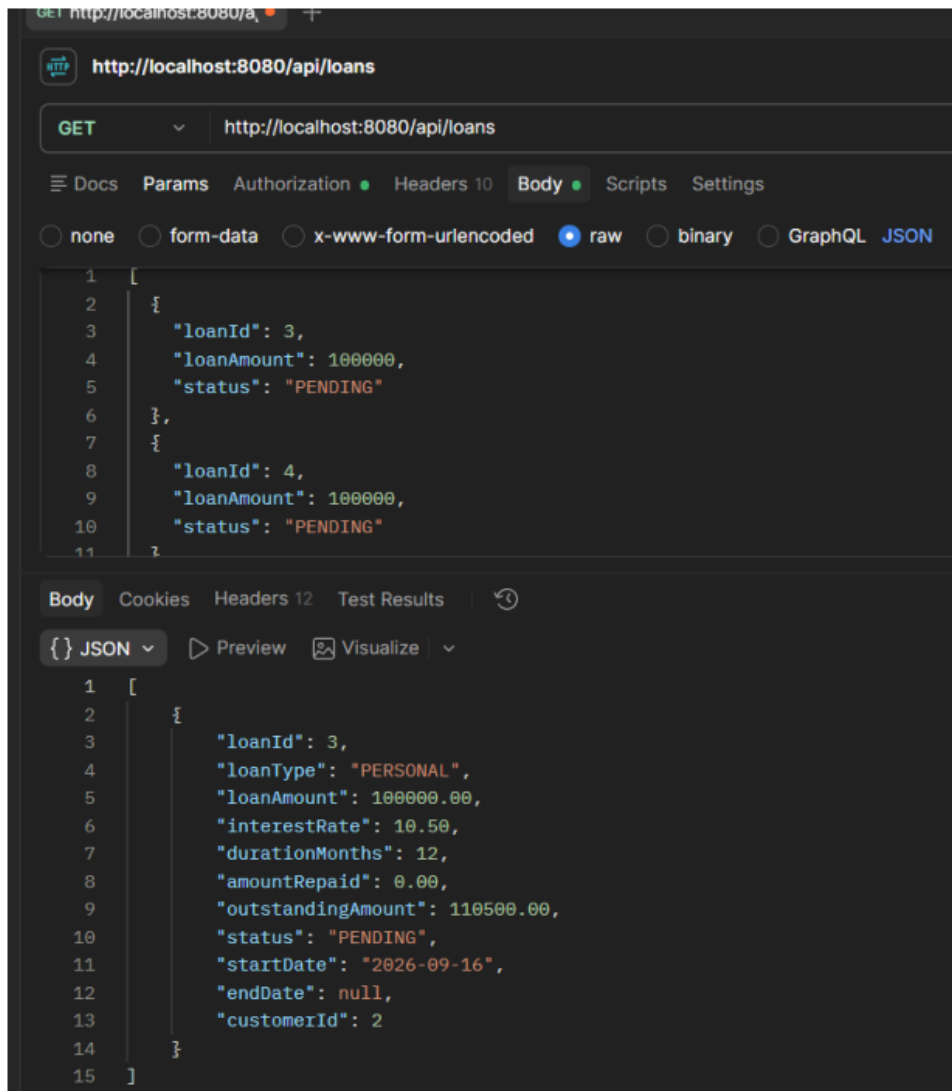
Delete :<http://localhost:8080/api/loans/2>

- **Approve Loan**

Get: <http://localhost:8080/api/loans>

Loan CRUD – Delete and Approval

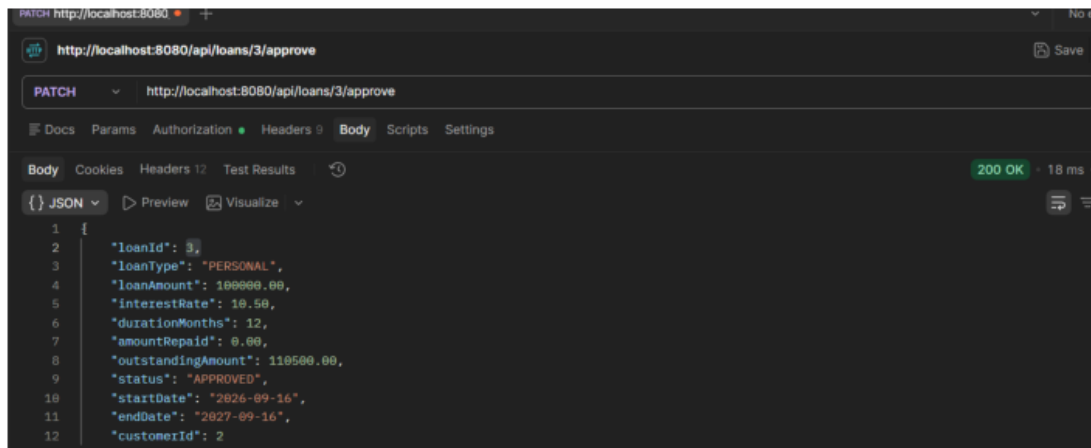
A DELETE request is shown for removing a loan. The page then introduces the loan approval operation, which is performed separately through the approval endpoint.



Patch: <http://localhost:8080/api/loans/3/approve>

Loan Approval

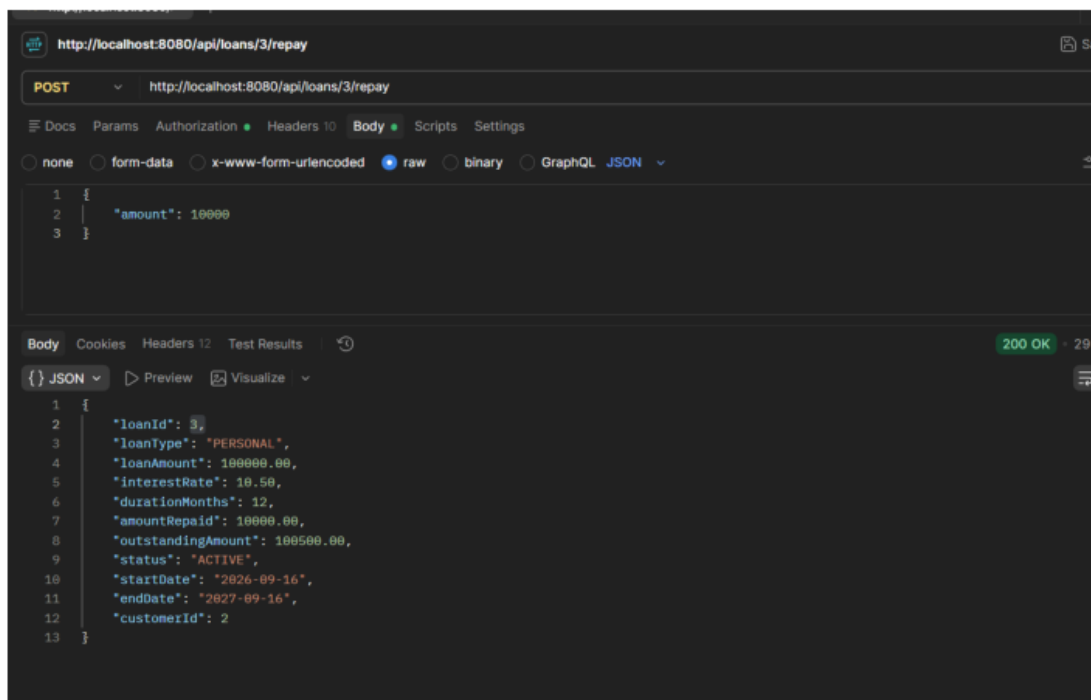
The PATCH `/api/loans/3/approve` endpoint is used to approve a pending loan. The Postman response shows the loan after the approval operation and provides evidence that the approval API executed successfully.



Loan Approval is successful.

- **Loan Repayment**

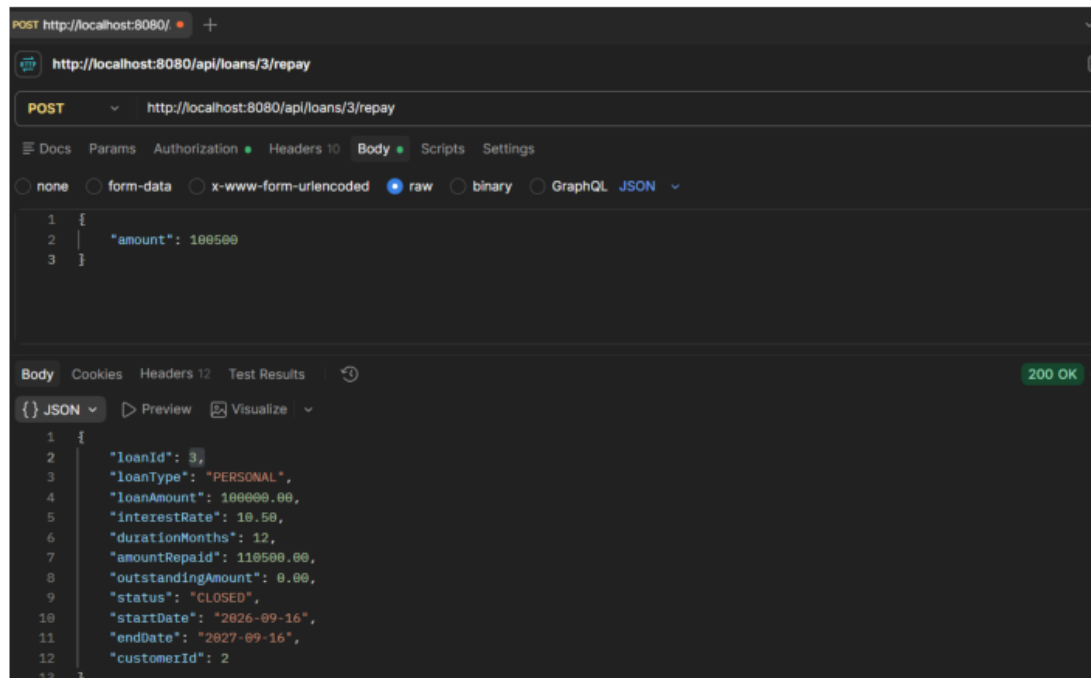
Post: <http://localhost:8080/api/loans/3/repay>



- **Full Loan Repayment**

Loan Repayment

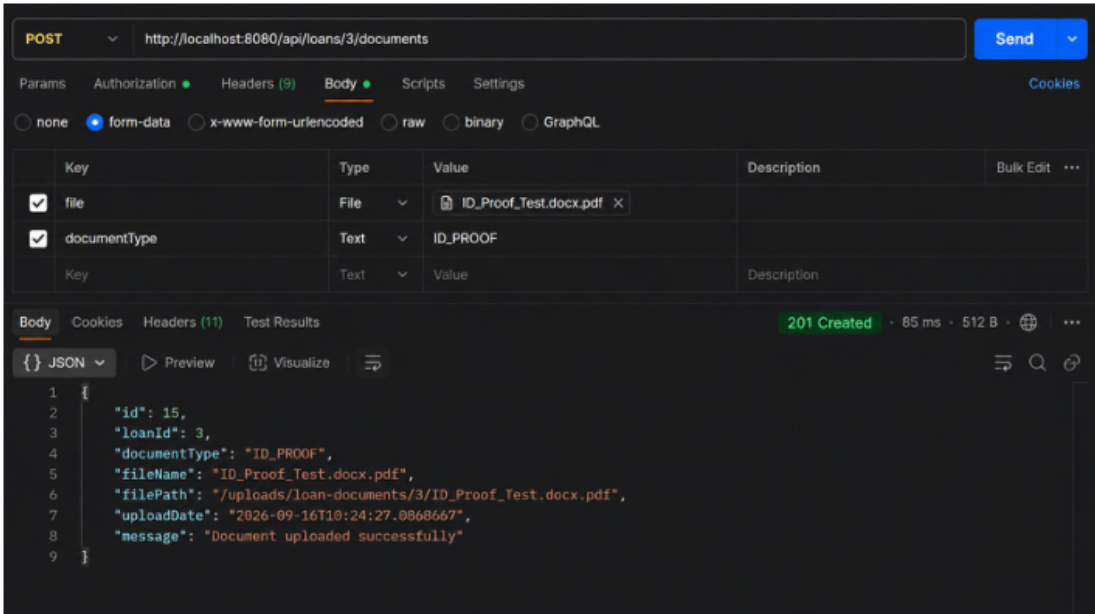
The loan approval result is followed by the repayment operation. The POST `/api/loans/3/repay` endpoint is used to submit a repayment amount for the approved loan.

Post: <http://localhost:8080/api/loans/3/repay>

- **Loan Document Upload**

Post: <http://localhost:8080/api/loans/3/documents>**Full Loan Repayment and Document Upload**

The repayment response demonstrates the completed repayment state. The page then introduces the loan document upload endpoint, which is used for file handling associated with a loan.



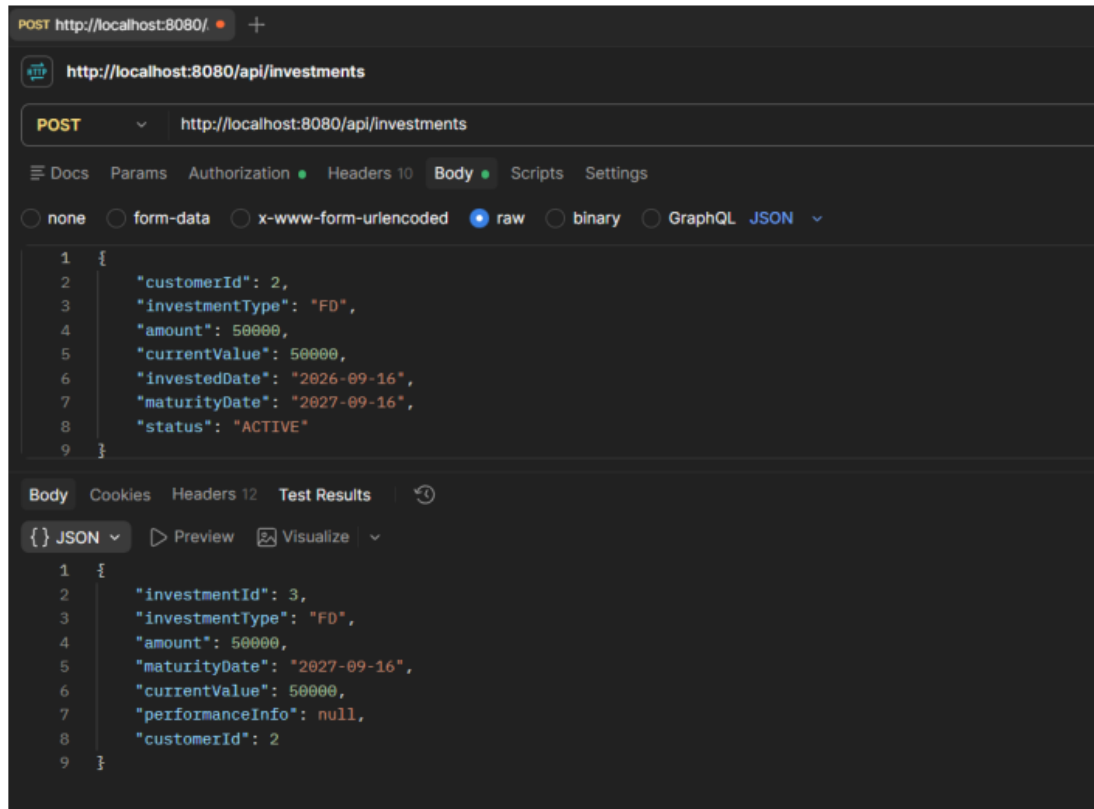
Successful Loan Document

- **Investment CRUD**

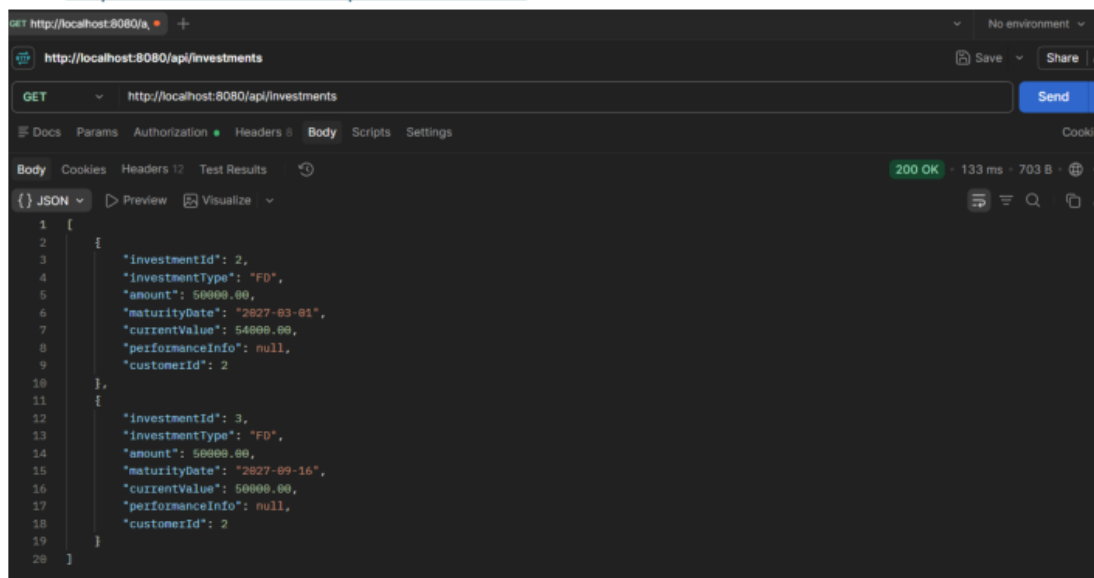
Create Investment

Loan Document File Handling – Upload

This screenshot shows Postman form-data being used to upload a loan document together with its documentType. The successful response contains document metadata such as document ID, loan ID, document type and file name. This provides direct evidence of the project's loan document file-handling feature.



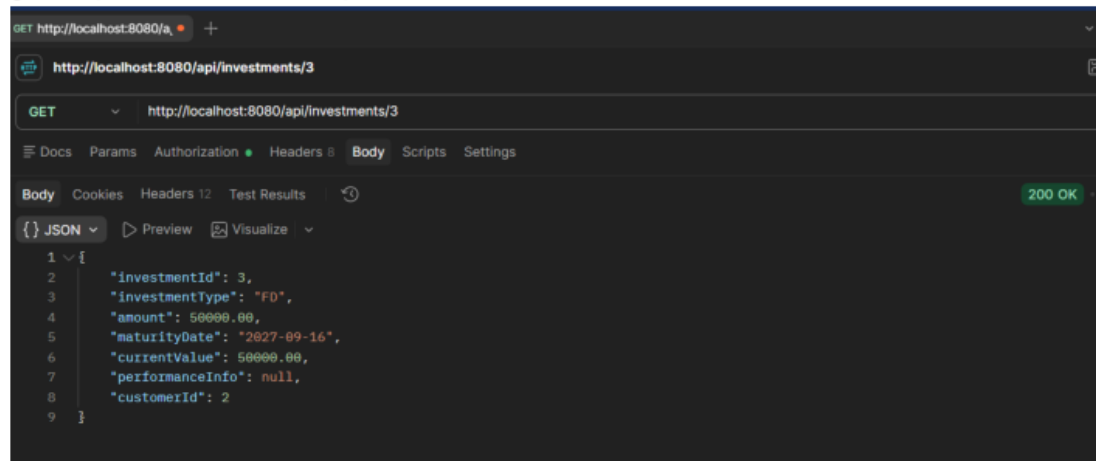
Get: <http://localhost:8080/api/investments>



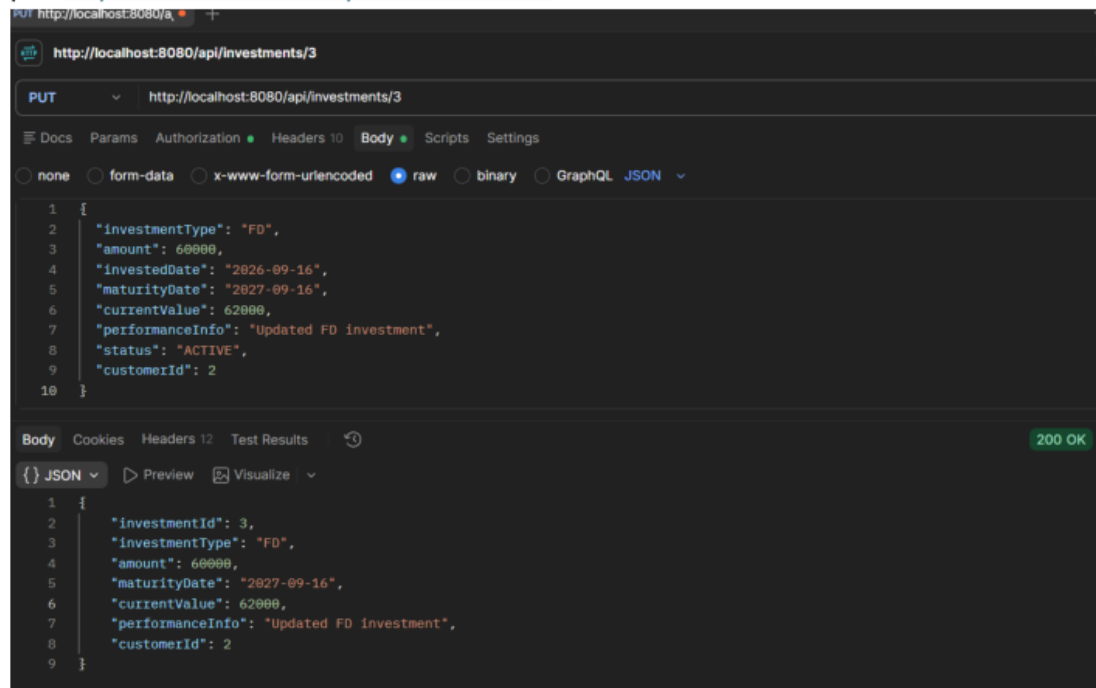
Investment CRUD – Create and Read All

The POST request creates an investment linked to a customer. The GET /api/investments request then retrieves investment records. The screenshots show the request and returned investment data for the Create and Read operations.

get: <http://localhost:8080/api/investments/3>



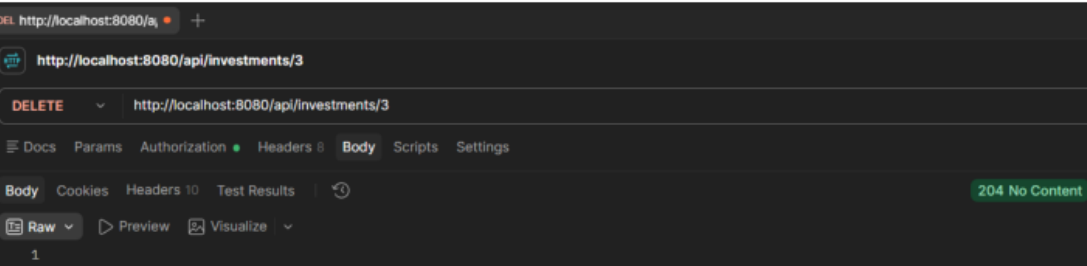
put: <http://localhost:8080/api/investments/3>



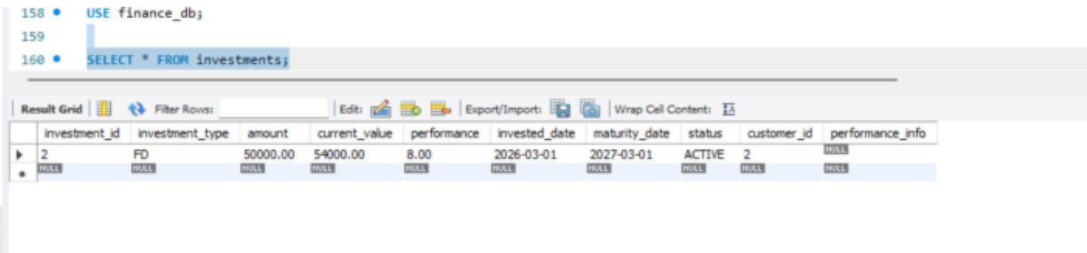
Investment CRUD – Read by ID and Update

The GET `/api/investments/3` request retrieves one investment by ID. The PUT request then updates the investment details, and the response shows the updated values. This demonstrates Read-by-ID and Update.

Delete: <http://localhost:8080/api/investments/3>



Database



- Final Project Verification

Investment CRUD – Delete and Database Verification

The DELETE /api/investments/3 request removes the selected investment. The database verification shown on this page checks the investment table after the operation. Final project verification begins here.

```

160 • SELECT * FROM investments;
161
162
163 • SHOW TABLES;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Tables_in_finance_db
accounts
customers
investments
loan_documents
loans
transactions

SELECT * FROM customers;

```

162
163 • SELECT * FROM customers;

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	customer_id	customer_name	email	phone	address	date_of_birth	created_at
▶	2	Krishna	krishna@gmail.com	9876543211	Bangalore	1999-08-20	2026-09-15 19:48:30
▶	4	Vamshi Krishna	vamshi2@gmail.com	9876543212	Hyderabad	2000-05-15	2026-09-15 23:41:18
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

SELECT * FROM accounts;

Final Database Verification – Customers and Accounts

The MySQL evidence shows the finance_db tables and SELECT queries for customers and accounts. This verifies that the main database tables exist and that records created or modified through the API can be inspected directly in MySQL.

160 •

SELECT * FROM investments;

161

162 •

SELECT * FROM accounts;

Result Grid

Filter Rows:

Edit:

Export/Import:

Wr

	account_id	account_number	account_type	balance	customer_id	created_at
▶	3	ACC709D537A6DB3	SAVINGS	8000.00	2	2026-09-16 01:06:25
*	NULL	NULL	NULL	NULL	NULL	NULL

SELECT * FROM transactions;

161

162 •

SELECT * FROM transactions;

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	transaction_id	transaction_type	amount	transaction_date	description	account_id	target_account_id
▶	2	WITHDRAWAL	2000.00	2026-09-16 01:17:10	NULL	3	NULL
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

SELECT * FROM loans;

161

162 •

SELECT * FROM loans;

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	loan_id	loan_type	loan_amount	interest_rate	total_payable	amount_paid	status	start_date	customer_id	amount_repaid	duration_months	end_date	outstanding_amount
▶	3	PERSONAL	100000.00	10.50	110500.00	0.00	CLOSED	2026-09-16	2	110500.00	12	2027-09-16	0.00
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

SELECT * FROM loan_documents;

162 •

SELECT * FROM loan_documents;

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	document_id	document_type	file_name	stored_name	file_type	file_size	file_path	uploaded_date	loan_id	content_type	original_file_name	stored_file_name	uploaded_at
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

SELECT * FROM investments;

Final Database Verification – Remaining Modules

The SELECT statements verify transactions, loans, loan_documents and investments. These queries provide database-level evidence for the major modules implemented in the project.

161 • `SELECT * FROM investments;`

investment_id	investment_type	amount	current_value	performance	invested_date	maturity_date	status	customer_id	performance_info
2	FD	50000.00	54000.00	8.00	2026-03-01	2027-03-01	ACTIVE	2	
*									

• Final API Verification

```

GET http://localhost:8080/api/customers

GET http://localhost:8080/api/customers

Body
JSON
[
  {
    "customerId": 2,
    "customerName": "Krishna",
    "email": "krishna@gmail.com",
    "phone": "9876543211",
    "address": "Bangalore",
    "dateOfBirth": "1999-08-28"
  },
  {
    "customerId": 4,
    "customerName": "Vamshi Krishna",
    "email": "vamshi2@gmail.com",
    "phone": "9876543212",
    "address": "Hyderabad",
    "dateOfBirth": "2000-05-15"
  }
]
200 OK

```

Final API Verification

The final Postman verification shows a GET request successfully returning customer records. Together with the preceding MySQL checks, this provides final evidence that the REST API and database are working together for the implemented project.